

INF2007 – TN4 – Melissa Moya

1. Approche et structure du programme

Le programme calcule la somme des sinus d'un tableau de 1 000 000 d'éléments (entiers ou flottants) via le flag `--type`. Deux fonctions spécialisées (`computeSineSumInt` / `computeSineSumFloat`) contiennent la boucle de calcul, tandis que `computeSineSum` dispatche via `interface{}` avec un surcoût négligeable (~1 ns) face à `math.Sin` (~20–40 ns). Pour garantir la reproductibilité, les tableaux sont générés avec `rand.NewSource(42)`. Bien que `rand.Intn(1001)` introduise un léger biais pour les plages non puissances de deux, celui-ci reste négligeable, car l'objectif est d'obtenir des données représentatives, non une distribution parfaitement uniforme. Côté architecture, `run()` retourne un struct `RunResult` sans affichage et `main()` gère seule la présentation des résultats, assurant une séparation nette entre logique et affichage.

Résultats complets, benchmarks et captures : [docs/](#) · [logs/](#).

2. Résultats des benchmarks

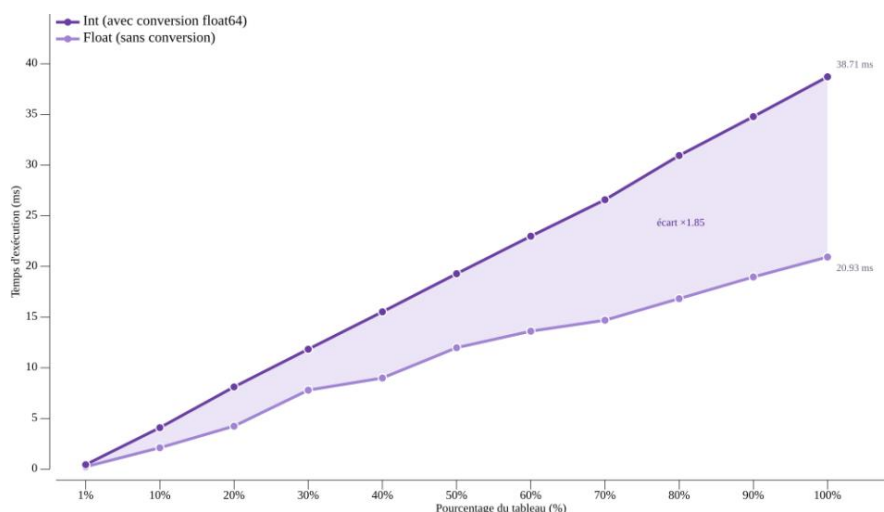
Les 22 sous-benchmarks (11 paliers × 2 types) ont été exécutés avec `go test -bench=. -benchmem -count=6` sur un Intel i5-10300H à 2,50 GHz (Windows/amd64, 8 threads) et analysés avec `benchstat` (médianes + IC 95 %). Pour isoler le pur coût de calcul, `b.ResetTimer()` exclut le setup, les tableaux sont pré-générés hors de `b.N`, et `b.ReportAllocs()` confirme 0 B/op, 0 allocs/op, ce qui indique que la performance est entièrement liée au calcul, sans aucune allocation mémoire. Le projet compte par ailleurs 13 tests unitaires avec une couverture de 100 %.

Tableau 1 – Temps de calcul par type et pourcentage du tableau

% du tableau	Éléments	Int (ms)	Float (ms)	Ratio
1 %	10 000	0.44	0.24	1.86×
10 %	100 000	4.09	2.11	1.94×
20 %	200 000	8.11	4.24	1.91×
30 %	300 000	11.83	7.79	1.52×
40 %	400 000	15.52	8.99	1.73×
50 %	500 000	19.28	11.98	1.61×
60 %	600 000	22.98	13.61	1.69×
70 %	700 000	26.58	14.69	1.81×
80 %	800 000	30.94	16.82	1.84×
90 %	900 000	34.78	18.96	1.83×
100 %	1 000 000	38.71	20.93	1.85×

Les flottants sont systématiquement plus rapides, avec un ratio de 1,5 à 1,9× selon le palier. De 50 % à 100 %, le temps double presque exactement (19,28 → 38,71 ms pour Int ; 11,98 → 20,93 ms pour Float), ce qui confirme bien la complexité $O(n)$. Aux paliers 90–100 %, l'intervalle de confiance atteint ± 1 %, signe de mesures stables.

Graphique 1 – Temps de calcul selon le pourcentage du tableau (Int vs Float)



La zone mauve translucide représente l'écart Int – Float ; l'annotation « $1.85\times$ » indique le ratio à 100 %. Pour régénérer le graphique, voir docs/chart-guide.md.

3. Analyse des résultats

Complexité et écart Int/Float.

Les flottants sont 1,5 à 1,9× plus rapides. L'écart provient de la conversion float64(v) à chaque itération et du coût de réduction d'argument de `math.Sin` pour les grands entiers, notamment `sin(500)` réduit ~ 79 tours modulo 2π , alors que les flottants dans $[0, 1)$ se situent déjà dans le domaine principal et ne requièrent aucune réduction. La variabilité aux paliers 30 % et 70 % (Float $\pm 7\%$ vs Int $\pm 1\%$) s'explique surtout par le bruit de mesure : les benchmarks Float étant plus courts, une même perturbation a un impact relatif plus visible.

Hiérarchie mémoire — L1/L2/L3.

L'i5-10300H dispose de L1 = 256 KB, L2 = 1 MB, L3 = 8 MB. Les petits paliers (1–10 %, 80–800 KB) tiennent entièrement en L2 ; le palier 30 % ($\sim 2,4$ MB) dépasse le L2 et sollicite le L3 ; le tableau complet ($1\,000\,000 \times 8$ octets = 8 MB) atteint exactement la limite du L3, limitant les accès par la bande passante, ce qui explique les légères non-linéarités aux paliers 70 %+.

Mécanismes micro-architecturaux (ch. 6).

`go build -gcflags=-m` confirme que `math.Sin` est inliné aux lignes 40 et 49 de `sinesum.go`, éliminant le surcoût d'appel et rendant le coût par itération strictement déterministe. La boucle `sum += math.Sin(v)` crée une dépendance séquentielle sur l'accumulateur, identique à l'exemple `prefixSum` du chapitre 6, qui empêche le pipeline de paralléliser les additions sur `sum`. Les appels `math.Sin(v)` pour des `v` différents sont indépendants et pipelinables (exécution dans le désordre), mais ce gain est limité par la latence intrinsèque de `math.Sin`. Utiliser plusieurs accumulateurs fusionnés en fin de calcul constituerait la principale piste d'optimisation.

4. Applications numériques

En prenant les médianes benchstat à 100 % du tableau, on obtient un temps par sinus de 38,7 ns pour les entiers (38 710 000 ns ÷ 1 000 000) et de 20,9 ns pour les flottants (20 930 000 ns ÷ 1 000 000).

Q1 — Distance parcourue par la lumière pendant un sinus ($c = 299\,792\,458$ m/s)

Type	Temps	Distance
Int	38,7 ns	$299,792,458 \times \frac{38,7}{10^9} \approx 11,6$ m
Float	20,9 ns	$299,792,458 \times \frac{20,9}{10^9} \approx 6,3$ m

Réponse : Pendant le calcul d'un sinus, la lumière parcourt environ 11,6 m avec des entiers et 6,3 m avec des flottants.

Q2 — Nombre de sinus par tick à 120 fps (tick = $\frac{1}{120}$ s = 8 333 333 ns)

Type	Temps/sinus	Sinus/tick
Int	38,7 ns	$8,333,333 \div 38,7 \approx 215\,333$
Float	20,9 ns	$8,333,333 \div 20,9 \approx 398\,726$

Réponse : À 120 fps, on peut calculer environ 215 000 sinus par frame avec des entiers et 399 000 avec des flottants.

Détails des calculs : [calculs.md](#)

5. Conclusion

Les mesures confirment que les flottants sont 1,85× plus rapides que les entiers, principalement en raison de la conversion float64(v) et de la réduction d'argument de math.Sin pour les grands entiers. La complexité $O(n)$ est confirmée empiriquement, et l'absence totale d'allocations mémoire (0 B/op) montre que la performance est entièrement liée au calcul. Ces résultats sont rendus fiables par l'utilisation de testing.B avec benchstat sur 6 exécutions (± 1 % aux paliers 90–100 %), approche bien supérieure à une simple mesure par time.Since. De plus, la vérification par go build -gcflags=-m confirme l'inlining de math.Sin, garantissant que les mesures reflètent le pur coût de calcul.

Dépôt GitHub github.com/moyamelissa/Advanced-Programming/tree/main/TN4